SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNETPROGRAMMING
COURSE CODE: CSA4303
Name: Duddela Harika
Reg No: 192324301

Total marks: 98

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

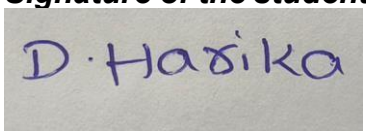
Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I Duddela Harika / 192324301 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginnin g)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	25
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningf ul analysis provided	25
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	19
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusion s	19
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	10

Answers: **Total: 98**

1. HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.

The given HTTP response header is:

HTTP/1.1			200			OK
Date:	Mon,	01	Apr	2026	10:00:00	GMT
Content-Type:						text/html
Content-Length:						-150
Connection: keep-alive						

The following issues can be identified:

1. Content-Length: The value -150 is invalid because the Content-Length field must contain a non-negative decimal integer representing the size of the response body in bytes.

2. Date: The weekday is incorrect. April 1, 2026, falls on a Wednesday, not a Monday. Therefore, the correct date is Wed, 01 Apr 2026 10:00:00 GMT.

3. Content-Type: The value text/html is valid and indicates that the response body contains HTML content.

4. Connection: The value keep-alive is valid for HTTP/1.1, although persistent connections are already the default unless otherwise specified.

5. Status line: HTTP/1.1 200 OK is syntactically valid. However, whether the status code accurately represents the result depends on the actual response and server processing.

2. Analyze the issue with Content-Length and explain its impact.

The Content-Length header specifies the size of the HTTP response body in bytes.

In the given response:

Content-Length: -150

The value -150 is invalid because the field cannot contain a negative number.

The impacts are:

1. Invalid response framing: The server provides an invalid body-length value, preventing the response from being correctly interpreted according to HTTP rules.

2. Browser processing problems: A browser or other HTTP client may reject the response, report a protocol error, or handle the invalid header differently.

3. Incomplete content delivery: If the response body cannot be framed correctly, the client may fail to display the expected HTML content.

4. Compatibility issues: Different HTTP clients and intermediaries may respond differently to the malformed header.

5. Potential security concerns: Inconsistent HTTP message parsing between servers and intermediaries can create security risks, particularly when message boundaries are interpreted differently.

The problem can be corrected by specifying the actual non-negative size of the response body in bytes. For example, if the body contains 150 bytes, the header should be:

Content-Length: 150

The value must match the actual number of bytes in the response body.

3. Determine whether the status code matches the response correctness.

The status code 200 OK indicates that the request was successfully processed and that the response is being returned successfully.

However, the given response contains an invalid Content-Length field.

The analysis is as follows:

1. 200 OK is a valid HTTP status code indicating success.

2. The invalid Content-Length: -150 makes the response header malformed.
3. Therefore, the response as a whole is not correctly formatted, even though the status code itself is valid.
4. A correctly implemented server should generate valid response headers and use a status code that accurately reflects the outcome.
5. If the server successfully processed the request and returned the intended content, 200 OK may be appropriate once the malformed header is corrected.

Conclusion: The status code is valid, but it does not make the malformed response header correct.

4. Suggest corrections for the identified errors.

The errors can be corrected by changing the invalid date and replacing the negative Content-Length value with the actual response-body size.

Corrected HTTP response header

HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

Note: The value 150 is an example. The actual Content-Length must equal the response body's size in bytes.

The following improvements should also be considered:

1. Correct the date: Use the proper weekday and date in the HTTP Date header.
2. Set the correct content length: Calculate the exact size of the response body in bytes.
3. Maintain a valid status code: Use 200 OK when the request succeeds and the response is valid.
4. Validate response headers: Ensure the server generates syntactically valid HTTP headers before sending the response.
5. Handle response framing correctly: When using HTTP/1.1, use valid message-framing rules. If appropriate, a server may use Transfer-Encoding: chunked instead of Content-Length, but it must not send both together in an ordinary HTTP/1.1 response.

Result

The HTTP response contains an invalid negative Content-Length value and an incorrect weekday in the Date header. Correcting these fields and ensuring that the Content-Length matches the actual response body makes the response properly formatted. The 200 OK status code can be retained if the request was successfully processed.

2: DOM Structure Analysis

Consider the following DOM tree representation

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

1. Identify missing or improperly closed elements.

The given HTML code is:

```
<html>
<head>
   <title>TestPage</title>
</head>
<body>
   <div>
      <p>HelloWorld
   </div>
</body>
</html>
```

The main error in this code is the missing closing tag for the paragraph element.

The errors are explained below:

1. Missing closing paragraph tag: The element is opened before the text HelloWorld, but the corresponding tag is missing. The paragraph should be closed explicitly.

2. Improper nesting: The closing tag appears before the paragraph is explicitly closed in the source. The paragraph should be closed before its parent element is closed.

3. Missing DOCTYPE declaration: The document does not contain <!DOCTYPE html>, which should be included to ensure that modern browsers use standards mode.

4. Missing language attribute: The <html> element does not specify the document language. Adding lang="en" helps browsers and assistive technologies identify the language.

5. Missing character encoding: The <meta charset="UTF-8"> element should be added inside the <head> section to specify the character encoding.

The <html>, <head>, <title>, <body>, and elements have corresponding closing tags. Therefore, the main

structural error is the missing tag.

2. Analyze how the browser will interpret this structure.

The Document Object Model (DOM) is a tree-like representation of an HTML document. It allows browsers and JavaScript programs to access and manipulate webpage elements.

When the browser receives the given HTML code, it parses the markup and constructs a DOM tree.

The browser processes the document as follows:

1. HTML parsing: The browser reads the HTML source from top to bottom and identifies the elements and text.
2. Head processing: It encounters the <head> element and identifies the page title as TestPage.
3. Body processing: It encounters the <body> element and begins processing the visible content.
4. Div creation: The browser creates a element inside the body.
5. Paragraph creation: The browser encounters and creates a paragraph element containing the text HelloWorld.
6. Automatic error recovery: When the browser encounters without an explicit closing , the HTML parser automatically closes the paragraph before closing the .
7. DOM construction: The browser constructs a DOM tree representing the corrected structure.

The resulting DOM will generally be equivalent to the following:

```
html
├── head
│   └── title
│       └── "TestPage"
└── body
    └── div
        └── p
            └── "HelloWorld"
```

This shows that the paragraph is a child of the element and contains the text HelloWorld.

Important: Modern browsers can recover from many HTML syntax errors. However, automatic recovery does not mean the original HTML source is correctly written. Developers should explicitly close elements and validate their code.

3. Determine the impact on rendering and layout.

The missing closing paragraph tag may not cause a major visible problem in this simple example because browsers automatically repair the markup. However, malformed HTML can cause problems in more complex webpages.

The impacts are explained below:

1. Unexpected DOM structure: Incorrectly nested elements can cause the browser to create a DOM tree that differs from the developer's intended structure.

2. CSS styling issues: CSS selectors depend on the document hierarchy. If the DOM structure is unexpected, styles targeting particular elements or relationships may not work as intended.

3. Spacing and alignment differences: Paragraphs have default browser margins. Unexpected element structures may affect spacing and the position of nearby content.

4. JavaScript functionality issues: JavaScript code often selects elements based on their parent-child relationships. If the DOM differs from the expected structure, scripts may select the wrong elements or fail to work correctly.

5. Browser compatibility concerns: Although modern browsers follow standardized HTML parsing rules, invalid markup can make debugging and testing more difficult across different environments.

6. Accessibility problems: A clear and predictable HTML structure helps screen readers and other assistive technologies interpret the webpage correctly.

7. Maintenance difficulties: Developers may find it harder to identify errors, modify content, or add new elements when the HTML structure is incomplete.

8. Validation errors: HTML validation tools may report errors or warnings when the source code does not follow the expected document structure.

For the given example, the text will generally appear inside the without a significant visual difference. Nevertheless, correcting the source is necessary to maintain reliable and understandable HTML.

4. Suggest corrections to make the DOM valid.

The code can be corrected by adding the missing closing </p> tag and including the appropriate HTML5 document structure.

Corrected HTML code

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>TestPage</title>
</head>

<body>

   <div>
```

```
    <p>HelloWorld</p>
  </div>

</body>
</html>
```

Explanation of the corrected code

- <!DOCTYPE html>: Declares the document as HTML5 and enables standards mode in modern browsers.
- <html lang="en">: Defines the root element and identifies English as the document language.
- <head>: Contains metadata and information about the webpage.
- <meta charset="UTF-8">: Specifies the character encoding for proper text display.
- <meta name="viewport">: Helps the webpage display appropriately on mobile devices.
- <title>TestPage</title>: Specifies the title shown in the browser tab.
- <body>: Contains the visible content of the webpage.
- : Groups the paragraph content.
- HelloWorld : Defines a paragraph with an opening and closing tag.

Correct DOM hierarchy

After parsing the corrected HTML, the browser creates the following DOM structure:

```
Document
└── html
    ├── head
    │   ├── meta
    │   ├── meta
    │   └── title
    │       └── "TestPage"
    └── body
        └── div
            └── p
                └── "HelloWorld"
```

This structure ensures that the paragraph is properly contained within the element and that the webpage follows standard HTML5 conventions.

Developers can also use the W3C Nu HTML Checker to identify missing tags, improper nesting, and other markup errors.

Result

The given HTML document contains a missing closing paragraph tag. Although modern browsers can automatically correct this error while constructing the DOM, relying on browser recovery can make the code harder to maintain. Adding the missing </p> tag, including the HTML5 document declaration, and using appropriate metadata produces a valid and predictable DOM structure, improving rendering, accessibility, and compatibility.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

1. Interpret the Differences in Data Transmission Between GET and POST

HTTP (Hypertext Transfer Protocol) is used for communication between a web browser and a web server. GET and POST are two common HTTP methods used to send data from a client to a server. However, they differ in how data is transmitted, their purpose, and their suitability for handling sensitive information.

GET Method

The GET method is mainly used to request or retrieve information from a server. When a form uses GET, its data is appended to the URL as query parameters.

Example:

GET /login?user=admin&pass=1234 HTTP/1.1
Host: example.com

In this example, the username admin and password 1234 are included directly in the URL. Anyone who can see the URL may be able to view these details.

Characteristics of GET:

- Form data is appended to the URL after a question mark (?).
- Data can be visible in the browser's address bar.
- URLs may be saved in browser history, bookmarks, server logs, and shared links.
- GET requests are generally intended for retrieving data.
- GET is suitable for search forms, filtering products, and retrieving public information.
- It is not suitable for transmitting passwords or other sensitive credentials.

POST Method

The POST method is generally used to submit data to a server for processing. Form data is transmitted in the request body rather than being appended to the URL.

Example:

POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

user=admin&pass=1234

Here, the username and password are sent in the request body, so they do not appear in the URL.

Characteristics of POST:

- Form data is sent in the HTTP request body.
- Submitted values are not normally displayed in the browser's address bar.
- POST is suitable for login forms, registration forms, and submitting information.
- It reduces exposure of credentials through URL history and copied links.
- POST does not encrypt data by itself; HTTPS is required to protect credentials during transmission.
- The server must validate the submitted data and verify the user's identity.

2. Analyze Which Method Is More Suitable for Login and Why

The POST method is more suitable for a login system because it sends the username and password in the HTTP request body instead of placing them in the URL.

A login form contains sensitive information that should not be unnecessarily exposed. Using GET can make credentials appear in the address bar, browser history, copied URLs, and server or proxy logs. This increases the possibility of accidental disclosure.

POST avoids putting credentials directly in the URL. However, it is important to understand that POST alone does not make a login system secure. The request body can still be exposed if the connection is not encrypted.

For a secure login system, POST should be combined with HTTPS, server-side validation, secure password storage, and appropriate session management.

Example of a suitable login form:

```
<form action="/login" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required>

  <br><br>
```

```
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>

    <br><br>

    <button type="submit">Login</button>
</form>
```

Explanation:

- The method="POST" attribute sends the form data in the request body.
- The action="/login" attribute specifies the server endpoint that processes the login request.
- The required attribute prevents the browser from submitting the form when either field is empty.
- The server must check the credentials against securely stored account information.
- The form must be submitted over HTTPS to protect credentials while they travel between the browser and server.

Client-side validation improves usability, but the server must independently validate all submitted data.

3. Identify Security Risks in Using GET for Authentication

Using GET for authentication can expose sensitive information and introduce several security risks.

a) Exposure of Passwords in URLs

When GET is used, credentials are included in the URL.

Example:

https://example.com/login?user=admin&pass=1234

Anyone who can access this URL may see the username and password. This is one of the main reasons GET should not be used for login forms.

b) Browser History Storage

Browsers may save URLs in browsing history. If a login URL contains a password, another person using the same device may discover the credentials by viewing the history.

c) Server and Proxy Logs

Web servers, reverse proxies, monitoring tools, and analytics systems may record request URLs. If passwords are included in query parameters, they may accidentally be stored in these logs.

d) Accidental Sharing of Credentials

Users may copy and share URLs while troubleshooting or sending links to others. If a URL contains authentication details, the credentials could be disclosed unintentionally.

e) Referrer Information Leakage

Depending on browser policies and application behavior, URLs containing sensitive query parameters may be sent as referrer information when a user navigates to another resource. This can expose information to other systems.

f) Increased Risk of Credential Theft

If an attacker gains access to browser history, logs, screenshots, or shared URLs, the exposed credentials may be used to access the account. This could result in unauthorized access and misuse of personal information.

g) False Sense of Security

A common mistake is assuming that POST is completely secure because its data is hidden from the address bar. In reality, an attacker may still intercept or read request data if the connection is unencrypted or the system is compromised.

Therefore, GET should not be used to submit passwords. POST reduces URL-related exposure, while HTTPS and secure server-side handling provide the essential additional protections.

4. Suggest Best Practices for Secure Login Implementation

A secure login system should follow multiple security practices rather than depending on only one HTTP method.

a) Use POST for Login Forms

The login form should use POST to submit usernames and passwords in the request body. Sensitive credentials should never be included in URL query parameters.

b) Use HTTPS

HTTPS encrypts communication between the browser and server using TLS. This protects credentials from being read or modified by attackers during transmission.

c) Validate Credentials on the Server

The server should verify the submitted username and password against the account records. It must not trust the browser alone, because client-side checks can be bypassed.

d) Store Passwords Securely

Passwords should never be stored as plain text. The system should use a password-hashing algorithm designed for this purpose, such as Argon2id or bcrypt, with appropriate parameters and salts.

e) Use Secure Session Management

After successful authentication, the server should create a secure session. Session cookies should use appropriate settings, including:

- Secure — sends the cookie only over HTTPS.
- HttpOnly — prevents ordinary client-side JavaScript from reading the cookie.
- SameSite — helps reduce certain cross-site request forgery (CSRF) risks.

The server should also invalidate sessions during logout and regenerate session identifiers when appropriate.

f) Add Login Attempt Protection

The system should use rate limiting or temporary delays to reduce automated password-guessing attacks. Suspicious login activity can also be monitored.

g) Display Safe Error Messages

The system should avoid revealing whether a particular username exists. For example, it can display:

Invalid username or password.

This reduces the information available to attackers.

h) Implement Multi-Factor Authentication

For accounts requiring additional protection, multi-factor authentication (MFA) can require a second verification step. This provides another layer of security if a password is stolen.

i) Avoid Logging Sensitive Information

Passwords, session tokens, and other secrets should not be written to application logs, analytics systems, or error reports. Logging should focus on useful security events without exposing credentials.

j) Protect Against CSRF Where Applicable

For cookie-based authentication, applications should use suitable CSRF protections, such as anti-CSRF tokens and appropriate SameSite cookie settings, where applicable.

Result

The differences between GET and POST were studied, and POST was identified as the more appropriate method for submitting login credentials because it avoids placing passwords directly in URLs. The security risks of GET, including exposure through browser history, logs, and shared links, were analyzed. Secure login practices such as HTTPS, server-side validation, password hashing, secure session management, rate limiting, and multi-factor authentication were recommended.

Conclusion: POST is preferable to GET for login forms, but a secure authentication system requires POST together with HTTPS and proper server-side security controls.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Identify Missing Closing Tags

The given snippet does not contain proper HTML tags for the heading, paragraph, and image. It also lacks the basic HTML document structure.

Issues identified:

- # Welcome is not a valid HTML heading. In Markdown, # represents a heading, but in HTML, headings must use tags such as <h1> or <h2>.
- This is a paragraph is plain text rather than content enclosed within a paragraph element.
- [image](image.jpg) is Markdown-style link syntax. It does not create an HTML image.
- The opening and closing <html>, <head>, and <body> elements are missing.
- The <!DOCTYPE html> declaration is missing.
- There are no explicit closing tags such as </h1> and because the corresponding HTML elements were never opened.

In HTML, most elements require matching opening and closing tags. For example, <h1>Welcome</h1> defines a heading, and This is a paragraph. defines a paragraph.

An image is different because <img> is a void element. It does not require a closing tag.

2. Analyze Issues With the Image Tag

The snippet contains [image](image.jpg), which is Markdown syntax for a link, not a valid HTML image element.

To display an image in HTML, the <img> element must be used.

Incorrect syntax:

[image](image.jpg)

This may be interpreted as ordinary text by an HTML parser. It will not reliably display the intended image.

Correct syntax:

<img src="image.jpg" alt="Description of the image">

Explanation of the attributes:

- img: Specifies that an image should be displayed.
- src="image.jpg": Specifies the path or URL of the image file.
- alt="Description of the image": Provides alternative text if the image cannot be displayed and helps screen-reader users understand the image.
- width and height: Can be added to specify the image dimensions and help the browser reserve space before the image loads.

Example:

<img src="image.jpg" alt="A beautiful landscape" width="400" height="250">

The image file must exist at the specified location. If it is stored inside an images folder, the source should be images/image.jpg. If the path is incorrect or the file is unavailable, the browser cannot display the intended image.

3. Determine How Browsers Will Handle These Errors

Web browsers use an HTML parsing process to interpret the document and construct the Document Object Model (DOM). Browsers are designed to recover from many HTML errors instead of immediately stopping page rendering.

The given snippet may be treated as text because the intended HTML elements are missing. The browser will not automatically understand that # Welcome is intended to be an HTML heading or that [image](image.jpg) is intended to display an image.

The browser's behavior can be explained as follows:

a) Heading Interpretation

The browser may display # Welcome as ordinary text. It will not apply the normal heading styles associated with an <h1> element.

b) Paragraph Interpretation

The text This is a paragraph will generally be displayed as ordinary text. Without a <p> element, it will not receive the normal paragraph structure and default paragraph spacing.

c) Image Interpretation

The Markdown-style text [image](image.jpg) will not be parsed as an HTML image. The image will not be displayed as intended unless valid HTML or a Markdown-processing system converts the syntax.

d) Missing Document Structure

When structural elements are omitted, browsers may create an HTML document structure implicitly during parsing. However, relying on automatic recovery is not recommended for well-structured webpages.

e) Rendering and Accessibility Issues

Incorrect markup can cause inconsistent styling, unexpected layout, and problems for assistive technologies. For example, a screen reader may not identify # Welcome as the page's main heading because it is not marked up with a heading element.

f) Validation Issues

HTML validation tools may report missing document structure and incorrect use of syntax. Validating the webpage helps developers identify mistakes before deployment.

4. Suggest a Corrected Version of the Code

The following is a corrected, standards-compliant HTML5 version of the snippet.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Welcome Page</title>
</head>
<body>

   <header>
      <h1>Welcome</h1>
   </header>

   <main>
      <p>This is a paragraph.</p>

      <img src="image.jpg"
         alt="A beautiful landscape"
         width="400"
         height="250">
   </main>
```

</body>
</html>

Explanation of the corrected code:

1. <!DOCTYPE html> tells the browser to use modern HTML standards and helps avoid quirks mode.
2. <html lang="en"> defines the document language as English.
3. <head> contains metadata and the page title.
4. <meta charset="UTF-8"> specifies the character encoding.
5. The viewport declaration supports responsive layouts on mobile devices.
6. <title> defines the title shown in the browser tab.
7. <header> contains the introductory heading.
8. <main> identifies the main content of the webpage.
9. <h1> correctly defines the main heading.
10. <p> marks the paragraph as a separate content block.
11. <img> displays the image using its source path and alternative text.
12. The closing tags ensure the document has a clear and maintainable structure.

Result: The HTML snippet was inspected, the missing structural elements and incorrect image syntax were identified, and a corrected HTML5 document was developed. The corrected code provides proper structure, improved accessibility, and more reliable browser rendering.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

1. Interpret Each Step in the Communication Flow

Step 1: User Enters a URL in the Browser

The process begins when the user enters a URL, such as:

https://www.example.com

A URL (Uniform Resource Locator) identifies the resource that the user wants to access.

It commonly contains the following components:

- https: The communication protocol.
- www.example.com: The domain name of the website.
- A path, such as /about: The specific resource requested, if included.

The browser analyzes the URL to determine the protocol, domain name, and resource path. It then determines whether it can reuse an existing connection or needs to establish a new one.

Step 2: DNS Lookup Occurs

DNS stands for Domain Name System. It translates a domain name, such as www.example.com, into an IP address that computers use to communicate over a network.

Humans can easily remember domain names, but network communication requires IP addresses.

For example:

Domain Name: www.example.com
IP Address: 93.184.216.34

The IP address above is illustrative; the actual address depends on the domain's DNS records.

The browser or operating system may first check its DNS cache. If the required information is not cached, a DNS query may be sent to a DNS resolver, which obtains the necessary record.

Once the address is resolved, the browser can connect to the appropriate server.

Importance of DNS:

- Converts domain names into IP addresses.
- Allows users to access websites using easy-to-remember names.
- Supports the distribution of traffic across different servers.
- Can reduce lookup delays through caching.

Step 3: HTTP Request Is Sent

After resolving the domain and establishing the required connection, the browser sends an HTTP request to the server.

HTTP stands for Hypertext Transfer Protocol. HTTPS is HTTP carried over a connection protected by TLS encryption.

A simple HTTP request may look like this:

GET / HTTP/1.1
Host: www.example.com
Accept: text/html

Explanation:

- GET: Requests a resource from the server.
- /: Specifies the requested path, in this example the website's root page.
- HTTP/1.1: Identifies the HTTP protocol version.
- Host: Identifies the requested domain.
- Accept: Indicates the types of content the browser can accept.

The request may also contain headers, cookies, authentication information, or a body, depending on the method and purpose of the request.

For HTTPS websites, the connection is protected by TLS, which helps protect data from being read or modified during transmission.

Step 4: Server Processes the Request

When the server receives the request, it determines how to handle it.

The server may perform several operations:

- Identify the requested resource.
- Check whether the requested page or file exists.
- Execute application logic.
- Validate user input or authentication information.
- Retrieve information from a database.
- Generate the required HTML or other response content.

For example, if a user requests a product page, the server may retrieve product information and prices from a database before generating the response.

If the resource exists and the request succeeds, the server may return 200 OK. If the requested resource is unavailable, it may return 404 Not Found.

The time required depends on server workload, application complexity, database queries, and other processing tasks.

Step 5: Response Is Returned

After processing the request, the server sends an HTTP response to the browser.

A simplified response might look like this:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<html>
<body>
   <h1>Welcome to Our Website</h1>
   <p>This page was returned by the server.</p>
</body>
</html>
```

Explanation:

- 200 OK: Indicates that the request succeeded.
- Content-Type: Identifies the type of content being returned.
- The response body contains the HTML content of the page.

The response may also include cookies, caching instructions, redirection information, or an error message.

The browser receives the response and processes the status code, headers, and response body.

Step 6: Browser Renders the Page

After receiving the HTML, the browser parses it and constructs the DOM (Document Object Model). It then processes CSS to determine the appearance and layout of the page.

JavaScript may also execute to provide interactive features.

The browser may request additional resources, such as:

- CSS stylesheets
- JavaScript files
- Images
- Fonts
- Videos and other media

These additional resources can require their own network requests. Once the browser has processed the necessary content, it displays the webpage to the user.

The browser may continue loading images, scripts, and other resources even after the initial content becomes visible.

2. Identify Where Delays Can Occur

Delays may occur at different stages of browser–server communication. Understanding these stages helps developers identify performance bottlenecks.

a) DNS Resolution Delay

If the domain name is not cached, the browser may need to wait for a DNS resolver to find the correct IP address. Slow DNS servers or network problems can increase this delay.

Improvement: Use reliable DNS infrastructure and suitable DNS caching.

b) Network Connection Delay

The browser must establish a connection with the server when an existing connection cannot be reused. Network distance, packet loss, and connection setup can increase the time required.

For HTTPS, TLS connection setup also contributes to the initial connection process, although modern protocols can reduce this overhead.

Improvement: Use connection reuse, efficient protocols, and reliable hosting.

c) Server Processing Delay

The server may take longer to respond if it performs complex calculations, handles too many requests, or executes inefficient application code.

Improvement: Optimize application logic, use caching, and scale server resources when necessary.

d) Database Query Delay

Web applications that retrieve data from databases may experience slow responses because of inefficient queries, missing indexes, excessive database calls, or overloaded database servers.

Improvement: Optimize queries, create appropriate indexes, and avoid unnecessary database operations.

e) Large Response Size

Large HTML documents, images, videos, and other files require more time to transfer over the network.

Improvement: Compress responses, resize images appropriately, and reduce unnecessary content.

f) Additional Resource Requests

A webpage containing many separate CSS files, JavaScript files, fonts, and images may need multiple network requests before it is fully displayed.

Improvement: Remove unused resources, bundle or optimize files when appropriate, and use caching.

g) Browser Rendering Delay

Complex CSS, large DOM structures, heavy JavaScript, and long-running scripts can delay rendering or make a webpage unresponsive.

Improvement: Optimize JavaScript execution, reduce unnecessary DOM elements, and prioritize important content.

Summary of Potential Delays

Stage	Possible cause	Improvement
DNS lookup	Slow DNS resolution	DNS caching
Connection setup	Network latency	Reuse connections
Request transmission	Slow or unstable network	Optimize network delivery
Server processing	Heavy workload	Application optimization
Database access	Slow queries	Indexing and query optimization
Response transfer	Large files	Compression and smaller assets
Browser rendering	Heavy scripts and styles	Optimize rendering

3. Analyze the Role of DNS in the Process

DNS plays a critical role in connecting a human-readable domain name to the IP address required for network communication.

Without DNS, users would often need to remember IP addresses instead of website names.

For example, when a user enters www.example.com, the browser needs to discover the IP address associated with that domain before it can connect to the appropriate server.

A simplified DNS resolution process is:

1. The browser checks its local DNS cache.
2. The operating system may check its own DNS cache.

3. If the answer is not cached, a DNS resolver is queried.
4. The resolver obtains the required DNS record, contacting other DNS servers if necessary.
5. The IP address is returned to the client.
6. The browser uses the address to connect to the server.

DNS records can also support load balancing and content delivery networks (CDNs), depending on the website's configuration.

Benefits of DNS:

- Makes websites easier to access.
- Separates memorable domain names from changing server addresses.
- Supports flexible server infrastructure.
- Can improve performance through caching.
- Helps distribute traffic across multiple servers or locations.

However, DNS does not transfer the webpage itself. It only helps the browser identify the network destination. The actual page content is obtained through subsequent HTTP or HTTPS communication.

4. Suggest Optimizations to Improve Performance

Website performance can be improved by optimizing DNS resolution, network connections, server processing, data transfer, and browser rendering.

a) Use DNS Caching

DNS caching stores previously resolved domain information for a period determined by the relevant caching rules. This can prevent repeated DNS lookups and reduce connection delays.

b) Use a Content Delivery Network (CDN)

A CDN stores copies of static resources, such as images, stylesheets, and JavaScript files, at servers distributed across different locations. Users can often download these resources from a nearby location, reducing latency.

c) Enable HTTP Compression

Text-based responses can be compressed using formats such as gzip or Brotli when supported. Smaller responses generally require less network transfer time.

d) Optimize Images and Other Media

Large images can slow page loading. Developers should resize images to suitable dimensions, use efficient image

formats, and avoid downloading unnecessary media.

e) Use Browser Caching

Caching allows browsers to reuse previously downloaded resources when the relevant HTTP caching rules permit it. This reduces repeated downloads when users revisit a website.

f) Optimize Server and Database Operations

Developers should improve inefficient application code, optimize database queries, use suitable indexes, and cache frequently requested data when appropriate.

g) Minimize JavaScript and CSS

Removing unused code and reducing file sizes can decrease download time. Deferring noncritical JavaScript and avoiding render-blocking resources can also help the main content appear sooner.

h) Use Modern HTTP Protocols

HTTP/2 and HTTP/3 provide improvements over older HTTP transport approaches. Depending on the browser, server, and network, these protocols can help improve resource delivery and connection efficiency.

i) Reduce Unnecessary Network Requests

Combining small resources where appropriate, eliminating unused scripts, and loading nonessential content only when needed can reduce page-loading overhead.

j) Monitor Performance Regularly

Developers can use browser developer tools, network waterfalls, Lighthouse, and Core Web Vitals to measure loading performance and identify bottlenecks.

Important metrics include:

- Largest Contentful Paint (LCP): Measures when the main visible content has loaded.
- Interaction to Next Paint (INP): Measures responsiveness to user interactions.
- Cumulative Layout Shift (CLS): Measures unexpected layout movement.

These measurements help developers understand how users experience the webpage and determine which optimizations are most useful.

Result

The browser–server communication flow was analyzed from URL entry and DNS resolution to HTTP request transmission, server processing, response delivery, and browser rendering. Potential delays caused by DNS lookup, network latency, server processing, database access, large files, and browser rendering were identified. Optimization

techniques such as DNS caching, CDNs, compression, browser caching, efficient database queries, and modern HTTP protocols were suggested.

Conclusion: Browser–server communication involves several interconnected steps. Optimizing each step helps reduce loading time, improve responsiveness, and provide a better user experience.